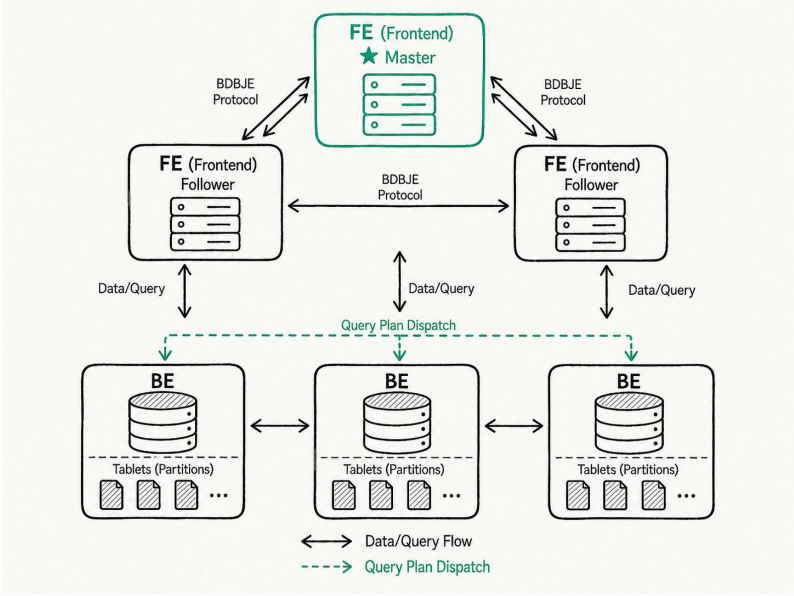
试题

- 1. Doris|Starrocks架构与原理相关
  - 1.1 Doris/ Starrocks的整体架构介绍（必看，高频面试题）
  - 1.2 存算分离架构和存算一体架构该如何选择？（艾特尔信息）
  - 1.3 Doris/StarRocks 在数据湖架构中的应用和优势是什么？（智汇创想）
  - 1.4 Doris/Starrocks的FE节点的选举机制
  - 1.5 介绍下Doris的数据分布、数据存储（虾皮）
  - 1.6 Doris/StarRocks的分布式事务是如何实现的?
  - 1.7 Doris/StarRocks的存储引擎有哪些特点？（虾皮）
  - 1.8 Doris/Starrocks 如何的FE/BE是如何保证数据一致性的？（飞轮科技）
  - 1.9 介绍一下BE的半数写入机制（飞轮科技）
  - 1.10 介绍一下Doris/Starrocks里面的version？（飞轮科技）
  - 1.11 为什么不用doris做引擎，而使用doris做数仓？（华盛通-深圳）
- 2. Doris|Starrocks核心特性与性能相关
  - 2.1 介绍下Apache Doris/ Starrocks的特点（必看，极其高频面试题）
  - 2.2 Doris / StarRocks 为什么会查询这么快？（信泰福建科技公司/金格方/麦可思数据科技/中国平安/钛动科技/星火游戏/58同城/360/快手/京东）
  - 2.3 ClickHouse 的 Join 性能为什么比starrocks/doris 差？（阿里巴巴/飞轮科技）
  - 2.4 Doris/Starrocks是如何实现高性能的多维分析?
  - 2.5 Doris/Starrocks 与 Clickhouse有什么区别？（必看，极其高频面试题）
  - 2.6 Doris/Starrocks 有哪些机制来保证数据的高可用？（卓繁信息）
  - 2.7 Doris/Starrocks和Hive对比说说有啥区别？（中科软/中数智汇/科大讯飞/恒创嘉越/吉贝克-海尔外包 ）
  - 2.8 doris是什么特性能让你们在项目中把ADS层放到doris，解决你们实际业务什么问题（南京365网络）
  - 2.9 doris/starrocks缺点有哪些？（携程/杭州消金）
  - 2.10 介绍Doris为啥比Spark快（花茎教育）
  - 2.11 starrocks和Doris对比（畅唐网络/Aftership/枫叶互动）
  - 2.12 Doris 版本相关问题（慧策）

1. Doris|Starrocks架构与原理相关

1.1 Doris/ Starrocks的整体架构介绍（必看，高频面试题）

（1）存算一体架构



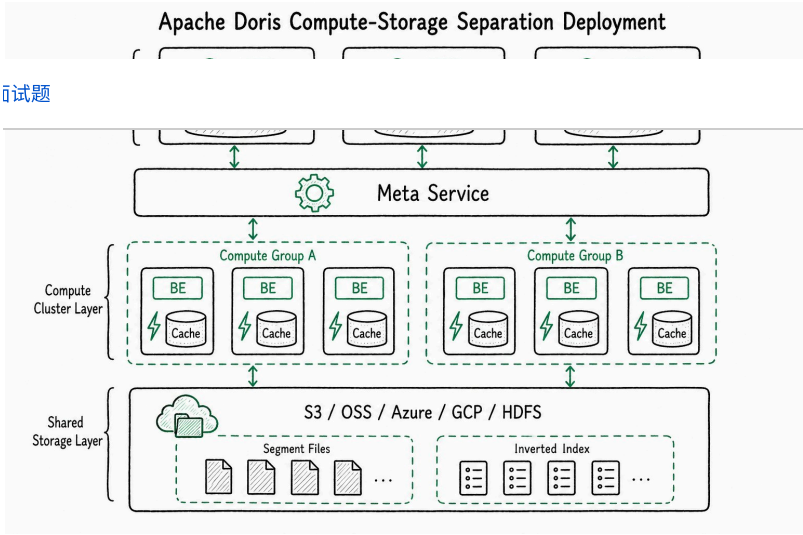
采用 **“FE（前端）+ BE（后端）”** 分布式架构：

- **FE**：负责元数据管理、SQL 解析优化、查询规划及集群调度。
- **BE**：承担数据存储、查询执行（计算）及数据副本管理。

整体介绍

- **设计**：计算与存储在同一 BE 节点，数据本地磁盘存储 + 本地并行计算。
- **优势**：数据本地性强，无网络 IO 开销，查询性能极致；架构简洁，运维成本低。
- **技术细节**：数据按 “分片（Tablet）” 分布式存储，多副本保障高可用；BE 通过 MPP 并行框架 + 向量化引擎执行计算。
- **适用**：数据规模稳定、对亚秒级查询要求高的场景（如实时业务监控、核心报表）。

（2）存算分离架构



- **元数据层：** 负责请求规划、查询解析以及元数据的存储和管理。
- **计算层：** 由多个计算组组成。每个计算组可以作为一个独立的租户承担业务计算。每个计算组包含多个无状态的 BE 节点，可以随时弹性伸缩 BE 节点。
- **存储层：** 可以使用 S3、HDFS、OSS、COS、OBS、Minio、Ceph 等共享存储来存放 Doris 的数据文件，包括 Segment 文件和反向索引文件等。

整体介绍

- **优势**：存储 / 计算可独立扩缩，云原生弹性好；利用对象存储实现海量数据低成本存储。
- **技术细节**：BE 查询时从对象存储拉取数据块，部分场景引入本地缓存层优化 IO 延迟。
- **适用**：PB 级超大规模数据、突发计算需求的云原生场景（如互联网用户行为分析）。

1.2 存算分离架构和存算一体架构该如何选择？（艾特尔信息）

在选择 Doris/StarRocks 的存算一体或存算分离架构时，可从**业务需求、资源弹性、部署复杂度**三个维度决策：

（1）存算一体架构

- **特点**：计算与存储在同一节点，部署简单，无外部存储依赖，数据本地性强，查询性能极致。
- **适用场景**：业务数据规模稳定、对弹性扩缩容要求低，追求极致查询性能的场景（如企业内部核心报表分析、数据量增长平缓的业务）。

（2）存算分离架构

- **特点**：计算与存储解耦，存储依赖共享存储（如 S3、OSS），计算资源可弹性伸缩，云原生适配性强。
- **适用场景**：数据量爆发式增长、需动态调整计算资源（如互联网用户行为分析、流量波动大的业务），或追求存储与计算独立扩缩的云原生场景。

简言之，若业务**追求性能稳定、部署简单**，选存算一体；若业务**需弹性扩缩、适配云原生**，选存算分离。

1.3 Doris/StarRocks 在数据湖架构中的应用和优势是什么？（智汇创想）

Doris/StarRocks 在数据湖架构中主要作为**湖仓一体的分析引擎**，实现对数据湖的高效查询分析，核心应用与优势如下：

（1）直接对接数据湖的存储系统（如 HDFS、S3、OSS 等），支持 Parquet、ORC 等主流数据湖格式，无需将数据从湖中迁移，即可对海量数据进行多维度分析查询，实现 “湖仓一体” 的高效分析能力。

（2）关键优势

- **无缝整合**：兼容数据湖多格式（Parquet、ORC 等）与存储系统（HDFS、对象存储等），直接查询湖中数据，无数据迁移成本。
- **实时分析**：依托 MPP 架构和向量化执行引擎，提供亚秒级至秒级查询响应，支持数据湖场景下的实时决策分析（如用户行为实时洞察）。
- **简化 ETL**：直接查询湖中原始数据，减少传统 ETL 的 “抽取 - 转换 - 加载” 步骤，降低数据处理复杂度与时间成本。
- **成本优化**：结合数据湖的低成本存储（如对象存储）与 Doris/StarRocks 的高效查询能力，实现 PB 级数据的经济化分析。
- **弹性扩展**：支持水平扩缩容计算节点，可根据数据湖数据量的增长灵活调整计算资源，适配业务的爆发式数据需求。

1.4 Doris/Starrocks的FE节点的选举机制

FE 节点分为 follower 和 observer 两类。各个 FE 之间，通过 bdbje 进行 leader 选举，数据同步等工作。Doris 将 FE 元数据存储在嵌入式的 BDBJE 数据库中，并通过其 **高可用复制机制（HA Replication）** 实现多节点一致性和主节点选举。

初始化复制组

- 启动时，每个 `Follower` FE 进程会尝试加入指定的复制组（通过配置中的 `priority_networks`, `edit_log_port`, `rpc_port` 等字段标识自己）。
- 所有 `Follower` 会自动发现并连接其他 FE 节点，组成一个 BDBJE 的 **Replication Group**。

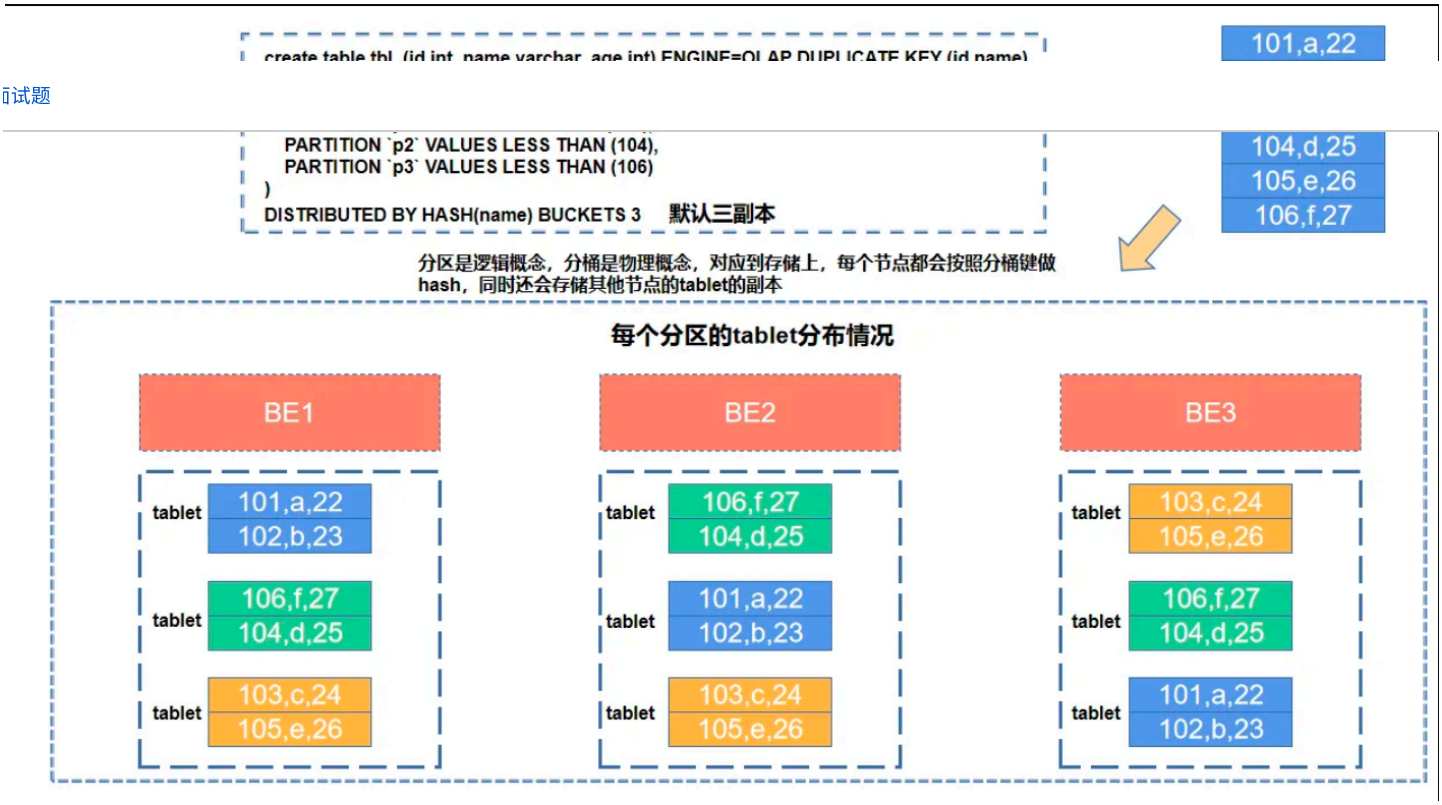
选举规则（基于 Paxos算法）

- BDBJE 使用类 Paxos 算法来从所有 `Follower` 节点中选出一个 **Master**（observer不参与选举）。
- 拥有多数节点可用（即 `>N/2` 个节点存活）时才可进行选举。

1.5 介绍下Doris的数据分布、数据存储（虾皮）

Doris的数据分布是在建表的时候就需要预先定义的，数据分布的好与坏同查询性能强相关。Doris提供分区分桶的选项供使用者对数据分布进行定义。

首先看一下底层存储文件tablet和分区分桶以及副本的关系



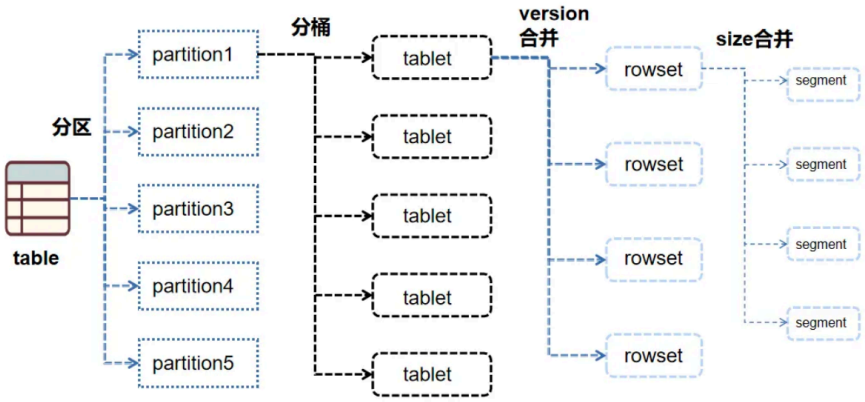
上图可以看到，每个分区内部，会按照分桶键将数据整体分成多个桶，同时根据副本数的配置，决定有多少个副本在其他节点上。当BE节点数和副本数一致时，每个BE节点会保留这个表对应的所有的tablet的数据**（自身hash分桶对应的tablet数据和其他节点tablet的副本）**，如上图一样。

因此 tablet数量的计算如下所示：**AllTabletNum = BucketsNum * PartitionNum * ReplicaNum**

对应上图的计算就是**3 * 3 *3 = 27 tablets**

表，分区，分桶，tablet，rowset，segment文件的关系

关系图如下所示：



数据在导入时，最先落到磁盘上的是当前导入批次生成的rowset文件，每一次导入事务的成功，都会生成一个带有版本号的rowset文件。rowset文件由多个segment文件组成，segment文件是由segment writer生成的文件块。合并生成的rowset文件会有一个带有起始index的version号，没有经过合并的rowset文件的起始索引号时一样的。然后根据相应的合并策略，满足这些合并策略的rowset文件会根据version号再次进行合并，生成tablet文件，供查询使用。

1.6 Doris/StarRocks的分布式事务是如何实现的?

StarRocks实现分布式事务的机制主要基于以下几个方面：

1.两阶段提交：采用两阶段提交协议来保证分布式事务的一致性。在第一阶段，事务协调器询问所有参与节点是否可以提交事务；如果所有节点都同意，那么在第二阶段进行实际的提交操作。

2.事务日志： 每个事务操作都会记录在事务日志中，这些日志用于在发生故障时恢复事务状态，保证事务的持久性和一致性。

3.版本控制：使用多版本并发控制（MVCC）来管理数据版本，这样可以支持在读取数据时不锁定数据，优化并发性能。

4.故障恢复： 在发生节点故障时，能够通过事务日志和保存的状态信息来恢复未完成的事务，确保数据一致性不受影响。

5.分布式共识协议： 使用分布式共识协议(Raft)协议保证分布式系统中数据的一致性。只有大多数副本写入成功，事务才会被认为是提交成功的

补充：

（1）多版本并发控制(MVCC):

- MVCC 允许多个事务并发执行，而不会直接阻塞读取操作。每个数据行都有多个版本，每个版本带有一个时间戳或事务标识。

- 长时间运行的查询可以读取特定版本的数据，使其不受并发写操据的影响
- 事务提交时，新版本的数据才会对其他事务可见，这样不会造成读到不一致的状态。

试题

- Raft 通过选举机制选举出一个Leader节点，所有的写操作都通过Leader节点来进行，由Leader节点确保数据一致地传播到其他Follower节点。
- 如果Leader节点发生故障，通过重新选举机制，系统确保可以快速选出新的Leader，从而保证系统的可用性和一致性。

1.7 Doris/StarRocks的存储引擎有哪些特点？（虾皮）

Doris/StarRocks 的存储引擎具备以下核心特点，适配 OLAP 分析场景的高性能需求：

（1）列式存储架构：采用**列式存储**方式，仅读取查询所需列数据，大幅减少 IO 开销，特别适合多维度分析场景。

（2）多数据模型支持：

- 明细模型：存储原始明细数据，支持任意维度分析。
- 聚合模型：数据导入时预聚合存储，减少查询时计算量，适合高频聚合查询。
- 更新模型：支持数据实时更新，满足有更新需求的分析场景（如用户标签更新）。
- starrocks还有专门为更新场景的primary key模型

（3）高效索引体系：支持 **Bitmap 索引、Bloom Filter、倒排索引** 等，快速过滤数据，缩小扫描范围，加速条件查询。

（5）分布式存储与高可用

- 数据按分片（Tablet）分布式存储，多副本机制保障节点故障时的数据可用性。
- 分片在集群节点间均衡分布，支持水平扩展，满足海量数据存储与并行查询需求。

（5）压缩和编码： 数据在存储时会进行压缩和编码，以减少存储空间并提高I/O效率。

（6）版本管理： StarRocks的存储引擎支持多版本并发控制（MVCC），这允许对数据进行非锁定读取，同时进行写入操作。

1.8 Doris/Starrocks 如何的FE/BE是如何保证数据一致性的？（飞轮科技）

FE数据一致性：FE 采用**类 Raft 协议（BDB JE 协议）**（Doris/StarRocks 均用）保证元数据强一致，只有 Leader 可写，变更同步到多数 Follower 才提交，确保所有 FE 元数据完全一致。导入/更新/删除以**事务**为单位，要么全部成功可见，要么全部失败回滚，**无中间状态。**

BE数据一致性：每个数据分片有多个副本，BE 之间通过 **Raft 日志同步 + Quorum 多数派写入**，确保至少半数副本写入成功才返回成功，保证副本数据完全一致。BE 写入采用 **WAL 预写日志 + 批量提交**，数据先写日志再落盘，机器宕机重启后可恢复，保证**单节点写入不丢、不乱**。

FE 定期检查 BE 副本状态、数据版本、分片完整性，发现副本缺失 / 版本不一致时，自动发起**副本补齐、版本同步**，保证全局数据一致。

1.9 介绍一下BE的半数写入机制（飞轮科技）

BE 的半数写入机制，就是**多副本写入时必须满足多数派（>1/2）成功才算成功**，既保证数据强一致、不丢失，又提升写入性能和可用性，是 Doris/StarRocks 高可靠的核心机制。比如一个 Tablet 数据默认 **3 副本，**写入时必须保证 **至少 2 个副本成功（> 半数）**。只要满足 **多数派成功**，就向 FE 返回事务成功，剩下不足的副本，**后台异步补齐**，不影响写入延迟。

1.10 介绍一下Doris/Starrocks里面的version？（飞轮科技）

在 Doris 和 StarRocks 里，**Version 就是数据的版本号**，由 FE 统一分配、全局单调递增，每次导入或更新事务提交都会生成一个新版本。

它主要用来实现 **MVCC 多版本并发控制**，保证读写互不阻塞、查询能拿到一致的快照数据，不会读到未提交或中间状态的数据。同一主键的多条数据会靠 Version 区分新旧，**Version 更大的就是最新数据**，在 Unique 表和主键表做数据合并、去重覆盖时，就是以 Version 作为判断依据。同时 Version 也用于分布式事务和多副本一致性，只有所有副本都同步到对应版本，数据才对外可见，从而保证整个集群的数据一致。后台 Compaction 也是按 Version 来合并小文件、清理旧版本，既减少文件数量提升查询性能，又能回收存储空间。

1.11 为什么不用doris做引擎，而使用doris做数仓？（华盛通-深圳）

Doris 本质是 **MPP 分析型数据库（实时数仓）**，不是通用计算引擎；用它做数仓是发挥一体化存储 + 计算 + 查询的整体优势，只当引擎用会浪费其他能力、架构更复杂。Doris 自带分布式存储、列式引擎、元数据管理、高可靠集群，而且支持**实时摄入（秒级）、Upsert 更新、多表 Join、物化视图、高并发等**，完整覆盖数仓 ODS→DWD→DWS→ADS 全链路。目前，市面上有非常多的公司使用Doris做数仓案例，非常成熟。完全不需要再额外增加新的组件，以免带来不必要的运维麻烦。

2. Doris|Starrocks核心特性与性能相关

2.1 介绍下Apache Doris/ Starrocks的特点（必看，极其高频面试题）

Apache Doris/ Starrocks是一款聚焦**现代实时数据分析需求**的数据库系统，融合 MPP（大规模并行处理）与 OLAP（在线分析处理）技术优势，以 “简化数据分析流程、提升数据查询速度” 为设计理念，专为海量数据的快速查询场景打造。典型场景：

1. **实时数据分析**典型场景：在线广告效果实时归因、电商实时流量统计等。依赖 Doris 实时数据摄取与分析能力，为业务提供即时决策依据。
2. **企业级数据仓库**凭借强大的查询性能与简洁的操作界面，成为企业 “主数据仓库” 的优选，支撑多维度、复杂逻辑的数据分析需求。
3. **BI 报表与可视化**作为商业智能（BI）工具的后端数据引擎，通过 SQL 接口提供高效数据查询能力，赋能业务人员通过可视化报表挖掘数据价值。
4. **用户行为分析**互联网企业可基于 Doris 实时分析用户行为数据（如点击、留存、转化等），为产品迭代、运营策略制定提供数据支撑。

技术特点：

1. **列式存储**采用列式存储方式，仅需加载查询所需列的数据，大幅减少 IO 操作，是 OLAP 场景下提升查询效率的关键设计。
2. **向量化执行引擎**摒弃传统 “逐行处理” 模式，实现数据批量化计算，显著提升计算速度与并发处理能力。
3. **智能索引技术**结合 Bitmap 索引与倒排索引，针对维度数据查询进行深度优化，进一步缩短查询响应时间。
4. **分布式架构**数据在多节点间分布式存储，查询任务并行执行，既支持水平扩展以应对数据量增长，又保障了大数据场景下的高性能。
5. **生态兼容性**提供与 MySQL 兼容的 SQL 接口，用户无需学习新查询语言即可快速上手；同时具备易部署、易管理的特性，降低运维成本

也可以整理下回复：为什么选Doris/了解过Doris吗？这种问题，结合下面的2.2 一起回复

2.2 Doris / StarRocks 为什么会查询这么快？（信泰福建科技公司/金格方/麦可思数据科技/中国平安/钛动科技/星火游戏/58同城/360/快手/京东）

试题

（1）整体架构：MPP架构 + 向量化执行：Doris 和 StarRocks 都是典型的 **MPP（Massively Parallel Processing）架构**，查询时会把 SQL 拆分成多个并行的 fragment，在各个 BE 节点上并发执行。

（2）存储层：列式存储 + 数据裁剪

Doris 和 StarRocks 的存储层都是基于 **列式存储**，同类数据连续存放，天然利于向量化计算。

性能核心点在于：

- **列裁剪：**查询只读取需要的列，其他列完全不扫描。
- **谓词下推（Predicate Pushdown）：**过滤条件直接在存储层生效，能尽量减少 IO。
- **前缀/Zone Map / Bloom Filter / MinMax 等索引：**存储文件中带有列值范围（Min/Max），快速跳过不相关数据块。

（3）执行层：向量化执行引擎

Doris / StarRocks 的查询执行是向量化的，也就是一次处理一批数据，不像传统火山模型那样一行一行处理。

优势：

- **CPU Cache 命中率高**（批量操作避免频繁函数调用）；
- **SIMD 指令并行加速**（利用 CPU 指令集同时处理多条数据）；
- **减少函数调用栈消耗**；
- **算子计算优化**（多个算子批量执行）。

（4）Pipeline 执行模型

新版本的 Doris和 StarRocks都引入了 **Pipeline 执行模型**，这是一个巨大的性能提升点。

模型	调度方式	并发性	缺点
Volcano（传统模型）	pull 模式，一层层取数	低（串行算子执行）	调度开销大、CPU idle 多
Pipeline（新模型）	push 模式，算子流水线并行	高（多算子同时执行）	调度更复杂

Pipeline 模型允许不同算子之间异步并发执行，减少线程切换和等待，每个 pipeline 被划分为多个 task，能更好利用多核资源。

（5）算子优化与代价模型（CBO）：StarRocks / Doris 在优化器层面引入了 **CBO（Cost-Based Optimizer），Doris还有RBO （Rule-Based Optimization），**这些优化器能够充分利用了丰富的数据统计信息，自动的尽可能的采用最优的策略来优化查询。

2.3 ClickHouse 的 Join 性能为什么比starrocks/doris 差？（阿里巴巴/飞轮科技）

ClickHouse 的 Join 性能相对较差，主要是由于其架构设计和功能特性的局限性。以下是 ClickHouse 在 Join 性能方面的劣势，以及 Doris/starrocks 的优势对比：

架构设计限制：

- ClickHouse 采用两层汇聚的执行框架，缺乏完善的 MPP 查询层，对复杂查询的支持较差，在大表之间的 Shuffle Join 场景下表现不佳
- 相比之下，Doris/starrocks 基于 MPP 架构，拥有完善的执行框架和向量化执行引擎，能够支持复杂的 SQL 分析，包括大小表的 Broadcast Join 和大表之间的 Shuffle Join

Join 类型支持有限：

- ClickHouse 仅支持 Global Join（对应Broadcast Join）和 Local Join（对应Colocation Join），在大表 Join 大表时，需要将右表广播到所有节点，性能受限
- Doris/starrocks 支持四种 Join 方式，包括 BroadCast Join、Shuffle Join/Bucket Shuffle Join 和 Colocation Join，能够根据数据规模和查询类型选择最优的 Join 策略

优化器和索引不足：

- ClickHouse 缺乏基于成本的优化器（CBO），无法对 Join 操作进行有效的优化。其聚合能力采用 Scatter Gather 模式的 2 阶段聚合，第二阶段为单节点，对内存和 CPU 的消耗较大
- Doris/starrocks 拥有较完善的 CBO，能够根据数据分布和查询计划选择最优的执行方式。其聚合能力采用多节点并行聚合计算，结合 Runtime Filter 技术，进一步提高 Join 性能

内存和数据处理能力：

- ClickHouse 的 Join 默认倾向于使用 Hash Join，要求右表能够完全加载到内存中构建哈希表。当右表过大时，会尝试将数据溢写到磁盘，导致性能显著下降
- Doris/starrocks 在设计上考虑了更复杂的 OLAP 场景，通过分区分桶裁剪、点查索引（主键、倒排）等技术，进一步提高并发度和 Join 性能

2.4 Doris/Starrocks是如何实现高性能的多维分析?

Doris（StarRocks 架构逻辑类似）主要通过以下技术手段实现高性能多维分析：

- **列式存储优化**：采用**列式存储**，多维分析时仅读取查询所需列，大幅减少 IO 开销，提升查询效率。
- **高效索引支持**：提供 Bitmap、Bloom Filter、倒排索引等**动态索引**，快速定位数据，缩小扫描范围，加速多维度过滤查询。
- **预聚合（物化视图）**：通过**物化视图**实现预聚合，数据导入时提前计算并存储聚合结果，查询时直接复用预计算结果，减少实时计算量，显著提升多维度聚合查询速度。
- **复杂查询优化机制**：内置**谓词下推、列剪裁、Join 优化**等机制，例如谓词下推可提前过滤数据，降低后续计算的数据集规模，提升多维关联查询性能。
- **灵活数据模型适配**：支持**明细、聚合、更新**等多种数据模型，可根据多维分析场景（如明细探查、多维度聚合）灵活选择，保障查询效率。
- **分布式 MPP 架构**：采用 MPP 的分布式架构，支持水平扩展，查询任务在多个节点并行执行，满足海量数据下的多维分析性能需求。

2.5 Doris/Starrocks 与 Clickhouse有什么区别？（必看，极其高频面试题）

Doris、StarRocks 和 ClickHouse 都是主流的 MPP 架构 OLAP 引擎，核心区别主要体现在架构设计、SQL 能力、查询性能、数据更新、易用性和适用场景上：

ClickHouse 是单一节点性能极强的列存引擎，单表查询、写入吞吐、数据压缩非常不错，适合日志、监控、时序这类大宽表场景，但多表关联能力弱、SQL 兼容性一般、运维和扩缩容比较复杂，也不擅长频繁实时更新。

Doris/Starrocks 架构非常简单，而且完全兼容 MySQL 协议和语法，开箱即用、运维简单、支持在线扩缩容，同时提供明细、聚合、Unique 多种数据模型，实时更新和多表 Join 能力明显优于 ClickHouse，而且并发能力很强，综合性能均衡，非常适合实时数仓、BI 分析、数据中台，日志等场景。

2.6 Doris/Starrocks 有哪些机制来保证数据的高可用？（卓繁信息）

主要通过以下机制保障数据高可用：

面试题

- 元数据冗余存储：元数据在多个节点间冗余存储，避免元数据单点故障，保障元数据的高可用与一致性。
- 多种数据恢复机制：支持备份/跨集群数据同步等方式，可在实际生产故障后，通过备份恢复数据，降低业务中断风险。

2.7 Doris/Starrocks和Hive对比说说有啥区别？（中科软/中数智汇/科大讯飞/恒创嘉越/吉贝克-海尔外包 ）

Hive 是构建在 Hadoop 之上的**离线数仓引擎**，采用存算分离架构，数据存储在hdfs或者s3上，依赖 MapReduce 或 Spark 执行计算，查询延迟高，以批量 ETL、数据清洗和离线统计为主，事务与实时更新能力较弱。

Doris 是**MPP 架构的实时分析数仓**，存算一体、自带分布式存储（Doris3.0之后支持存算分离，数据可以存储在hdfs或者s3上，主要是为了节省成本），采用向量化执行引擎，查询延迟低、并发能力强，支持实时写入与 Upsert 更新，更适合交互式分析和高并发报表服务。

延伸的话，也可以和spark对比下（科大讯飞问过）

2.8 doris是什么特性能让你们在项目中把ADS层放到doris，解决你们实际业务什么问题（南京365网络）

把 ADS 层放到 Doris，主要利用它**低延迟查询、高并发、MySQL 兼容、实时写入更新**的特性。既能支撑**高并发报表、大屏、自助分析**等毫秒级响应需求，又能直接对接实时数据做到**T+0 实时指标**，同时运维简单、稳定性强。解决了传统方案查询慢、并发低、实时性差、组件繁多难维护的问题，实现了**统一的 ADS 查询服务**。

2.9 doris/starrocks缺点有哪些？（携程/杭州消金）

整体在实时分析场景表现优异，但仍存在短板。在**存算一体架构**下，存储与计算绑定，弹性扩缩容不灵活，容易出现资源浪费，且大查询、高并发场景下易出现内存压力大、OOM 等问题。引入**存算分离**后，虽然解决了弹性与存储成本问题，但会带来性能损耗，查询延迟和吞吐不如存算一体稳定，对网络和共享存储依赖极高，架构也更复杂、运维难度更大。同时，Doris/Starrocks 对复杂嵌套类型、半结构化数据支持较弱，不擅长长时批处理与重度 ETL，多表大关联仍需谨慎设计，周边生态工具链相比 Hive 等传统组件也不够完善，多租户隔离能力相对有限。

2.10 介绍Doris为啥比Spark快（花茎教育）

Doris 采用 MPP 全内存并行 + 向量化执行引擎，查询无需落盘（4.0版本之后支持落盘，可选），而 Spark 基于批处理，中间结果频繁读写磁盘，IO 开销极大。同时 Doris 支持智能索引、前缀过滤、Runtime Filter 等优化，大幅减少数据扫描与网络 Shuffle，整体延迟比 Spark 低一个数量级，在多维分析与多表关联场景下速度优势尤为明显。

2.11 starrocks和Doris对比（畅唐网络/Aftership/枫叶互动）

一、Apache Doris 与 StarRocks 详细系统对比

1. 起源与 License

- Doris：源自百度 Palo，Apache 顶级项目，Apache 2.0 协议，完全开源、无商用限制。
- StarRocks：基于早期 Doris 分支自研，Linux 基金会，早期 Elastic 协议，现转为 Apache 2.0，但企业版高级功能闭源。

2. 核心架构（几乎同源）

- 均为 FE（元数据）+ BE（计算存储） 、MPP + 向量化执行引擎、RAFT 高可用、分区 + 分桶分布式存储。
- 都支持存算分离和存算一体的架构

3. 功能差异

（1）数据模型与更新能力

- Doris：支持 Unique / Aggregate / Duplicate 三种模型；支持灵活列更新、高并发写入、Group Commit、时序 Compaction，高频导入性能更稳定。
- StarRocks：比Doris多一个 Primary Key 模型，支持行更新与部分列更新；无灵活列更新，不支持时序 Compaction。

（2）半结构化与多模态支持

- Doris：原生支持 Variant 类型、JSON、倒排索引、IK/ICU/ 自定义分词；支持向量检索与10+ LLM 内置函数（分类、抽取等）。
- StarRocks：半结构化能力和分词能力有限，无 LLM 函数，向量检索能力弱。

（3）存储与冷热分层

- Doris：支持列存、行列混存、部分列行存；支持 HDD + SSD + 对象存储三级冷热分层。
- StarRocks：支持列存、行列混存；不支持部分列行存，冷热分层两级分层（HDD/SSD）。

（4）数据湖与湖仓一体

- Doris：支持 10+ 外部数据源，Iceberg 增量读取、Time Travel、Branch/Tag 能力完善。
- StarRocks：仅支持少量数据源，缺少增量读取、数据回溯等核心能力。

（5）高可用、容灾与运维

- Doris：支持跨集群复制（CCR）、磁盘间副本均衡、IO 隔离、大查询自动限流。
- StarRocks：不支持跨集群复制、不支持磁盘间均衡、不支持 IO 隔离，资源管控能力弱。

二、SelectDB、镜舟科技 与它们的关系

1. SelectDB（飞轮科技）：Apache Doris 商业公司，由 Doris 创始团队创立。是 Apache Doris 社区最大贡献者，提供企业版、云服务、技术支持。
2. 镜舟科技：StarRocks 的商业公司，由 StarRocks 核心团队创立，主导 StarRocks 开源迭代、企业版、商业化与生态建设。

2.12 Doris 版本相关问题（慧策）

更新时间：2026年6月26日（4.x的后面版本会变化，注意关注）

目前Doris的最新版本是4.1.2，稳定版本是4.0.6，这两个版本同步更新。（主流用4.x的不多）

以前有3.1.x的版本，最高到3.1.4，目前不更新了

3.0的版本，最高到3.0.8，目前不更新了

2.1.x 和2.0.x这都是很久远的版本了，大家可以说自己用3.1.x或者4.0.x。

Doris的版本稳定性判断，相对来说，前两位数一样的情况下，**第三位数越大越稳定**。

原因： Doris命名是x.x.x 的机制，第一位是大版本更新，频率很慢，年度才会更新，一般都是上一些很大的功能，比如存算分离，算子落盘，AI等。第二位版本是新的大一

[试题]

郑冬综合2502 2026/7/12 20:58:39
涤生大数据版权所有，请勿外传

无标签